Ders 6 Genişletilmiş Notu: Uygulama: Geriye Doğru Yerine Koyma ile Rekürans Çözümü ve Toplamların Büyüme Derecesi

Ders 6 Genişletilmiş Notu: Uygulama: Geriye Doğru Yerine Koyma ile Rekürans Çözümü ve Toplamların Büyüme Derecesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Uygulama Dersinin Amacı
- 2. Sonuçlarda Neden Θ Yazıyoruz?
- 3. Alıştırma 1: $x(n) = x(n-1) + n$
- 4. Alıştırma 2: $x(n) = x(n/3) + 1$
- 5. Alıştırma 3: $\sum_{i=2}^{n-1} \log_2 i^2$ Toplamının Büyüme Derecesi
- Örnek Sorular
- Dersin Toparlaması

1. Uygulama Dersinin Amacı

Bu kayıt, haftanın teorik dersinden sonra yapılan uygulama (problem çözme) saatidir. Hoca, o hafta öğrenilen konularla ilgili birkaç soru çözüleceğini söyledi. Yöntem şöyleydi: Hoca bir problemi ekrana yazar, öğrencilere 5-6 dakika süre verir, sonra çözümü birlikte yaparlar. İlk konu geriye doğru yerine koyma (backward substitution) yöntemiydi.

Önceki derslerde iki tür analiz görülmüştü: özyinelemeli olmayan (non-recursive) algoritmaların analizi ve özyinelemeli (recursive) algoritmaların matematiksel analizi. Özyinelemeli algoritmaların analizinde algoritmanın maliyeti bir rekürans bağıntısı (recurrence relation) olarak yazılır. Bu bağıntı, başlangıç koşuluyla (initial condition) birlikte geriye doğru yerine koyma yöntemiyle kapalı bir formüle çevrilir. Bu uygulama saatinde bu çözüm adımları üç alıştırma üzerinde çalışıldı.

2. Sonuçlarda Neden Θ Yazıyoruz?

Alıştırmalara geçmeden önce bir öğrenci, önceki derste çözülen örneklerle ilgili bir soru sordu.

Soru: Önceki örneklerde sonucu bulduktan sonra "$\Theta(\log n)$'in elemanıdır", ondan öncekinde "$\Theta(n)$'in elemanıdır" dedik. Neden Θ kullandık?

Cevap: Hoca önce "worst case ve best case" diye başladı, sonra bunun yanlış ifade olacağını söyleyip düzeltti. Doğru kavramlar üst sınır (upper bound) ve alt sınır (lower bound) idi. Derste problemleri matematiksel olarak analiz ederken üst sınırı ve alt sınırı ayrı ayrı ortaya koymadık. Bu problemlerde üst sınır ile alt sınır aynı noktaya denk geldiği için sonucu sıkı sınır (tight bound) olarak ifade edebildik. Bu yüzden Θ kullanıldı. Hoca, normalde bunun ayrıca değerlendirilmesi, bakılması gerektiğini söyledi. Derste böyle olduğu varsayıldığı için ifade bu şekilde yazılmıştı.

Ek bilgi (derste anlatılmadı): $O(g(n))$ bir üst sınır, $\Omega(g(n))$ bir alt sınır bildirir. Bir fonksiyon hem $O(g(n))$ hem $\Omega(g(n))$ kümesindeyse $\Theta(g(n))$ kümesindedir. Yani Θ yazmak, iki sınırın aynı fonksiyona denk geldiğini söylemektir. İki sınır farklıysa yalnız O ya da yalnız Ω yazılabilir.

3. Alıştırma 1: $x(n) = x(n-1) + n$

3.1 Problem ve Başlangıç Koşulu

Hoca ekrana şu bağıntıyı yazdı ve geriye doğru yerine koyma ile çözülmesini istedi:

$$x(n) = x(n-1) + n, \quad n > 0$$

Bir öğrenci hemen başlangıç koşulunun ne olduğunu sordu. Hoca bunu "çok güzel" diye karşıladı. Başlangıç koşulu verilmezse bu bağıntı geriye doğru yerine koymayla çözülemez. Çünkü elimizde yalnız rekürans bağıntısı vardır, yerine koymanın nerede duracağını ve oradaki değerin ne olduğunu söyleyen bir bilgi yoktur. Hoca bunun üzerine başlangıç koşulunu verdi:

$$x(0) = 0$$

Hocanın vurgusu: Başlangıç koşulu olmadan bir rekürans bağıntısı geriye doğru yerine koyma ile çözülemez.

Soru: Başlangıç koşulu normalde veriliyor mu, yoksa biz kendimiz mi seçiyoruz? Örneğin $x(1) = 1$ ile başlayabilir miyiz?

Cevap: Bu örnekte başlangıç koşulu verilmiştir, yani bağıntı ve başlangıç koşulu önceden hazırlanmıştır. Ama bir algoritmayı analiz ederken başlangıç koşulunu bizim bulmamız gerekir.

Ek bilgi (derste anlatılmadı): Bir özyinelemeli algoritmada başlangıç koşulu, algoritmanın kendini artık çağırmadığı durma noktasından (temel durum, base case) okunur. Örneğin $n = 0$ için hiç işlem yapmadan dönen bir fonksiyonda $M(0) = 0$ yazılır.

Soru: Çözümü adım adım mı yazmalıyız, yoksa yalnız cevabı mı vermeliyiz?

Cevap: Çözümün adım adım yazılıp sonuca ulaşılması gerekir.

Hocanın vurgusu: Yalnız sonucu yazmak yetmez. Geriye doğru yerine koymanın her adımı yazılmalıdır.

3.2 Adım Adım Çözüm

Hoca bu soruyu kendisi çözdü ve ilerleyen sorularda bunu öğrencilerden isteyebileceğini söyledi.

Birinci adım. Elimizde $x(n) = x(n-1) + n$ var. $x(n-1)$ yerine kendi karşılığını yazarız. Hoca bunun nereden geldiğini sordu. Öğrencilerin cevabı: Bağıntıda n gördüğümüz her yere $n-1$ yazarsak $x(n-1) = x(n-2) + (n-1)$ elde ederiz. Bunu yerine koyunca:

$$x(n) = [x(n-2) + (n-1)] + n = x(n-2) + (n-1) + n$$

İkinci adım. Aynı şekilde $x(n-2) = x(n-3) + (n-2)$ yazılır:

$$x(n) = [x(n-3) + (n-2)] + (n-1) + n = x(n-3) + (n-2) + (n-1) + n$$

Genel adım. i adım sonra ifade şu hali alır:

$$x(n) = x(n-i) + (n-i+1) + (n-i+2) + \cdots + n$$

Hoca bu genel adımı öğrencilere sordu. Bir öğrenci önce "$i \cdot n - i$" önerdi. Hoca bunun doğru olmadığını söyledi. Doğru yazım şöyle kuruldu: Toplamın ilk terimi $n - i + 1$, ikinci terimi

$n - i + 2$ olur ve terimler n'e kadar gider. Örneğin $i = 3$ için ilk terim $n - 2$ olur, bu da yukarıdaki ikinci adımla uyuşur.

Son adım. Başlangıç koşulunu kullanmak için $x(n - i)$ ifadesinin $x(0)$ olması gerekir. Bu yüzden $i = n$ konur:

$$x(n) = x(0) + 1 + 2 + 3 + \cdots + n = 0 + \frac{n(n+1)}{2} = \frac{n(n+1)}{2}$$

Sonuç olarak $x(n) = \frac{n(n+1)}{2} \in \Theta(n^2)$ bulunur. Ders sırasında bir öğrenci de sonucu "n çarpı $n + 1$ bölü 2" olarak söylemişti.

> **Ders sonrası not:** Hoca sonucu toparlarken bir yerde "$n+1$ bölü 2" dedi. Doğrusu $\frac{n(n+1)}{2}$'dir. Python ile $n = 0, \ldots, 29$ için bağıntı doğrudan hesaplandı ve hepsinde $\frac{n(n+1)}{2}$ ile aynı çıktı. Örneğin $x(10) = 55$.

3.3 Önceki Dersteki Faktöriyel Örneğiyle Farkı

Bir öğrenci şunu sordu: Önceki derste çözdüğümüz örnekte $M(n - i) + i$ çıkmıştı, i yerine n yazınca sonuç yalnız n olarak kalmıştı. Burada ise terimler $1, 2, 3, \ldots$ diye n'e kadar gittiği için $\frac{n(n+1)}{2}$ bulduk. Fark nereden geliyor?

Hocanın cevabı: İki rekürans bağıntısı farklıdır. Önceki dersteki bağıntı $M(n) = M(n-1)+1$ idi. Her adımda sabit bir 1 ekleniyordu. Bu yüzden i adım sonra $+i$ birikiyordu. Buradaki bağıntıda ise her adımda $+n$ ekleniyor. n yerine sırayla $n-1$, $n-2$ yazıldıkça eklenen terimler de değişir. Bu terimler $n - i + 1$ ile başlar ve n'e kadar devam eder. $i = n$ konunca bunlar $1, 2, 3, \ldots, n$ olur ve bu toplamın kapalı formu $\frac{n(n+1)}{2}$'dir. Kısacası öncekinde sabit bir sayı, burada n vardır.

4. Alıştırma 2: $x(n) = x(n/3) + 1$

4.1 Problem

İkinci örnek şuydu:

$$x(n) = x(n/3) + 1, \quad n > 1, \qquad x(1) = 1$$

Hoca çözerken $n = 3^k$ kabul edilebileceğini söyledi. Öğrencilere 5-6 dakika verildi. Hoca, çözümü elle yazıp fotoğrafını göndermelerini istedi, gelen çözümleri ekrana yansıtacağını söyledi. İki öğrencinin çözümü ekranda değerlendirildi.

4.2 Birinci Öğrencinin Çözümü: $n/3^i$ Üzerinden

Birinci çözüm doğrudan $n/3$ üzerinden ilerledi. $x(n/3)$ yerine $x(n/9) + 1$ yazıldı, sonra $n/3^3$, $n/3^4$ diye devam edildi. Her adımda bir $+1$ eklendi:

$$\begin{aligned} x(n) &= x(n/3) + 1 \\ &= x(n/3^2) + 1 + 1 \\ &= x(n/3^3) + 1 + 2 \\ &\;\;\vdots \\ &= x(n/3^i) + 1 + (i - 1) \end{aligned}$$

Öğrenci eklenen miktarı ilk $+1$ ve sonradan biriken $+(i-1)$ olarak ayrı yazmıştı. Logaritmaya geçmeden önce $n = 3^k$ ve $i = k$ konuldu. $3^k/3^k = 1$ olduğu için:

$$x(n) = x(1) + 1 + (k-1) = 1 + k = k + 1$$

Hoca, çözümde son satırın yazılmadığını, not olarak eklenmesi gerektiğini söyledi: k yerine $\log_3 n$ konur ve sonuç

$$x(n) = \log_3 n + 1$$

olur. Hoca bunun doğru bir çözüm şekli olduğunu söyledi.

4.3 İkinci Öğrencinin Çözümü: 3^k Üzerinden

İkinci çözüm, derste yapılan yönteme daha yakındı. Önce n yerine 3^k konuldu. $3^k/3 = 3^{k-1}$ olduğu için:

$$\begin{aligned} x(3^k) &= x(3^{k-1}) + 1 \\ &= [x(3^{k-2}) + 1] + 1 = x(3^{k-2}) + 2 \\ &\;\;\vdots \\ &= x(3^{k-i}) + i \end{aligned}$$

Hoca bu çözümde genel adımın eksik olduğunu söyledi. Arada mutlaka $x(3^{k-i}) + i$ satırı bulunmalıydı. Bu satır yazıldıktan sonra i yerine k konur:

$$x(3^k) = x(3^0) + k = x(1) + k = 1 + k$$

Burada $x(1)$'den 1 gelir. Geri kalan birler ise toplamda k tanedir: genel adımda $k-1$ tane ve dışarıda kalan bir tane daha. Bu yüzden $+k$ elde edilir. $k = \log_3 n$ konunca sonuç yine

$$x(n) = \log_3 n + 1 \in \Theta(\log n)$$

olur. İki çözüm de aynı sonuca ulaştı.

> **Hocanın vurgusu:** Geriye doğru yerine koymada genel adım (i adım sonraki ifade) mutlaka yazılmalıdır. Sonuca bu satırda i yerine durma değeri konarak geçilir.
>
> **Ders sonrası not:** Sonuç Python ile doğrulandı. $n = 1, 3, 9, 27, 81, 243$ için bağıntı doğrudan hesaplanınca $x(n) = 1, 2, 3, 4, 5, 6$ çıkar. Bu değerler $\log_3 n + 1$ ile aynıdır.

5. Alıştırma 3: $\sum_{i=2}^{n-1} \log_2 i^2$ Toplamının Büyüme Derecesi

5.1 Problem

Üçüncü örnek, hocanın ekranda gösterdiği "Exercise 3" idi:

$$\sum_{i=2}^{n-1} \log_2 i^2$$

Bu toplamın büyüme derecesi (order of growth) istendi. Öğrencilere yine 5 dakika verildi.

Hoca bu alıştırmada geriye doğru yerine koyma uygulanmadığını hatırlattı. Çünkü burada bir rekürans bağıntısı yoktur. Yalnız verilen toplam ifadesi açılarak büyüme derecesi bulunur. Bir öğrencinin limit kullanma girişimine de hoca "Limit kullanmaya gerek yok" dedi.

5.2 Öğrencilerden Gelen Cevaplar

Hoca, çözümü yapanlara cevaplarını sordu.

- Bir öğrenci cevabı $\log_2(n-1) \cdot (\log_2(n-1)+1)$ olarak verdi. Başka bir öğrenci de aynı sonucu bulduğunu, büyük ihtimalle eksikleri olduğunu söyledi. Hoca, gelen bir yazılı cevapta da bazı eksiklikler olduğunu belirtti.
- Bir öğrenci şöyle bir yol anlattı: Toplamı $n!$ ve n'e ayırdı, ikisini $\log n! - \log n$ olarak yazdı. Sonra önem derecesine göre $\log n$ terimini eledi. Çünkü $n!$ daha karmaşıktır. Ona göre karmaşıklık $\log_2 n!$ oluyordu.
- Bir öğrenci cevabın $n \log n$ olup olmadığını sordu. Hoca bunun doğru cevap olduğunu söyledi. Öğrenci gerekçeyi şöyle verdi: Logaritmaların toplamı için bir formül vardı, 1'den n'e kadar $\log i$ toplamı yaklaşık $n \log n$'e eşittir. Burada toplarsak yaklaşık $2(n-1)\log(n-1)$ çıkıyor, bu da $n \log n$'lik bir karmaşıklıktır.

  Ders sonrası not: $\log_2(n-1) \cdot (\log_2(n-1)+1)$ cevabı doğru değildir. Bu ifade $\Theta(\log^2 n)$ mertebesindedir. Oysa toplamda yaklaşık n tane logaritmik terim vardır, bu yüzden toplam $n \log n$ mertebesindedir. Python ile $n = 1000$ için toplam ≈ 17039, bu ifade ise ≈ 109 çıkar.

  Ders sonrası not: $2(n-1)\log_2(n-1)$ yaklaşımı toplamın tam değeri değil, bir üst sınırıdır. Çünkü her terim $\log_2 i^2 \leq 2\log_2(n-1)$'dir. Python ile $n = 1000$ için bu ifade ≈ 19909, toplam ≈ 17039 çıkar. Büyüme derecesi yine de aynıdır: $\Theta(n \log n)$.

  Ders sonrası not: $\log n! - \log n$ yolu doğrudur. Ancak sonuç $\log_2 n!$ olarak bırakılmamalı, bir adım daha gidilmelidir: $\log_2 n! \in \Theta(n \log n)$ olduğu için toplamın büyüme derecesi $\Theta(n \log n)$'dir.

5.3 Hocanın Çözümü

Birinci adım: Üssü dışarı almak. Hoca önce sınırlara dokunmadan i^2'deki 2'yi logaritmanın önüne aldı, sonra bu sabiti toplamın dışına çıkardı:

$$\sum_{i=2}^{n-1} \log_2 i^2 = \sum_{i=2}^{n-1} 2\log_2 i = 2\sum_{i=2}^{n-1} \log_2 i$$

İkinci adım: Sınırları değiştirmek. Hoca, toplamı daha önce verilen formüllerin biçimine getirmek istediğini söyledi. Bunun için sınırların değiştirilebileceği hatırlatıldı. Bir öğrenci yeni bir değişken tanımlamayı ($k = i - 2$ gibi) önerdi. Hoca başka bir değişken kullanmak istemediğini söyledi. Hedef, toplamı $i = 1$'den n'e giden $\sum \log_2 i$ biçimine getirmekti.

Hoca şunu sordu: Alt sınırı 1'e, üst sınırı $n-1$'den n'e çekersem denkleme başka ne eklemem gerekir? Alt sınırı 2'den 1'e indirmek $\log_2 1$ terimini ekler, ama bu 0'dır. Üst sınırı n'e çıkarmak $\log_2 n$ terimini ekler. Bu yüzden toplamdan $\log_2 n$ çıkarılmalıdır. Bir öğrencinin cevabı da buydu: "Eksi $\log n$ yapacağız."

$$2\sum_{i=2}^{n-1} \log_2 i = 2\left(\sum_{i=1}^{n} \log_2 i - \log_2 n\right)$$

Bir öğrenci logaritmayı doğrudan dönüştürerek de sınırların değiştirilebileceğini söyledi. Hoca farklı çözümlerin olabileceğini, doğru olduğunu, ama önce bu çözümün tamamlanacağını söyledi.

Üçüncü adım: Bilinen formülü kullanmak. Hoca ekranda maviyle çevrelediği $\sum_{i=1}^{n} \log_2 i$ kısmının derste daha önce görüldüğünü hatırlattı. Bu toplam yaklaşık $n \log n$'dir:

$$\sum_{i=1}^{n} \log_2 i \approx n \log_2 n$$

Çıkarılan terim ise $\log_2 n$'dir. Hoca burada neden $2 \log n$ değil de $\log n$ yazdığını sordu. Açıklaması şuydu: Bu ifadedeki sabitlerle işimiz yoktur. $\log_2 n$ teriminin karşılığı $\Theta(\log n)$'dir. Maviyle çevrili kısım ise matematiksel formülle $n \log n$'e denk gelir. 2 katsayısı bu $n \log n$'in dışında kaldığı için hoca onu ayrıca öne taşıdı. Hoca bunun çok önemli olmadığını, ama doğru gösterim bu olduğu için böyle yazdığını söyledi.

Dördüncü adım: Düşük dereceli terimi atmak. Elimizde $2 \cdot \Theta(n \log n)$ ve $\Theta(\log n)$ vardır. $n \log n$, $\log n$'den daha büyük olduğu için $\log n$'in artık bir önemi kalmaz ve elenir. Sabit de atılır. Sonuç:

$$\sum_{i=2}^{n-1} \log_2 i^2 \in \Theta(n \log n)$$

> **Hocanın vurgusu:** Toplamın büyüme derecesi bulunurken sabitler atılır ve daha düşük dereceli terimler elenir. $\Theta(n \log n)$ ile $\Theta(\log n)$ bir araya gelince sonuç $\Theta(n \log n)$ olur.
>
> **Ek bilgi (derste anlatılmadı):** $\sum_{i=1}^{n} \log i = \log(1 \cdot 2 \cdots n) = \log n!$'dir. $\log n! \in \Theta(n \log n)$ olduğu iki yönlü bir sınırla gösterilebilir. Üst sınır: her terim $\log i \leq \log n$ olduğu için toplam $\leq n \log n$'dir. Alt sınır: son $n/2$ terimin her biri $\geq \log(n/2)$ olduğu için toplam $\geq \frac{n}{2} \log \frac{n}{2}$'dir. Stirling yaklaşımı da aynı sonucu verir.
>
> **Ders sonrası not:** Python ile toplam doğrudan hesaplandı: $n = 10, 100, 1000, 10^5$ için toplam sırasıyla $\approx$ 36,9, 1036,2, 17038,9, 3033375,1. $2(\log_2 n! - \log_2 n)$ formülü $n \leq 1000$ için aynı değerleri verdi. Toplamın $n \log_2 n$'e oranı 1,11, 1,56, 1,71, 1,83 çıkar ve n büyüdükçe yavaşça 2'ye yaklaşır. Bu, büyüme derecesinin $\Theta(n \log n)$ olduğunu gösterir.

5.4 Çözümün Özeti

Bir öğrencinin isteği üzerine hoca çözümün üzerinden bir kez daha geçti:

- i^2'nin üssündeki 2 logaritmanın önüne geçirildi, sonra toplamın dışına alındı. Artık bütün ifadeye uygulanıyordu. İçeride yalnız $\log_2 i$ kaldı.
- Sınırlar $i = 2$'den $n - 1$'e kadardı. Alt sınır 2'den 1'e geri çekildi, üst sınır $n - 1$'den n'e çıkarıldı.
- Bunun karşılığında toplamdan $\log_2 n$ çıkarıldı. Bu terim doğrudan $\Theta(\log n)$ olarak yazıldı.
- Maviyle çevrili $\sum_{i=1}^{n} \log_2 i$ kısmı için dersin başında verilen formülden yaklaşık $n \log n$ elde edildi. Bütün toplama etki ettiği için 2 dışarıda kaldı. Bu sabit önemli değildir.
- $\Theta(\log n)$, $\Theta(n \log n)$'den daha küçük bir derece olduğu için elendi. Sabit de atılınca büyüme derecesi $\Theta(n \log n)$ olarak kaldı.

> **Ders sonrası not:** Hoca bu özetin sonunda bir an sonucu "Theta log n" olarak söyledi. Doğru sonuç $\Theta(n \log n)$'dir. Hoca da çözümün kendisinde ve öğrencinin cevabını onaylarken $n \log n$ demişti.

Örnek Sorular

Bu derste ekran karesi yoktur. Aşağıdaki sorular, derste sözlü olarak verilen ve ekrana yazılan alıştırmalardır.

Soru 1 (derste sözlü, 00:04:06)

$x(n) = x(n-1) + n$, $n > 0$ ve $x(0) = 0$ bağıntısını geriye doğru yerine koyma ile çözün.

> **Çözüm (derste verildi):** $x(n) = x(n-i) + (n-i+1) + \cdots + n$ genel adımında $i = n$ konur: $x(n) = x(0) + 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \in \Theta(n^2)$. Ayrıntı Bölüm 3'tedir.

Soru 2 (derste sözlü, 00:15:40)

$x(n) = x(n/3) + 1$, $n > 1$ ve $x(1) = 1$ bağıntısını $n = 3^k$ kabul ederek çözün.

Çözüm (derste verildi): $x(3^k) = x(3^{k-i}) + i$ genel adımında $i = k$ konur: $x(3^k) = x(1) + k = k + 1$. $k = \log_3 n$ olduğu için $x(n) = \log_3 n + 1 \in \Theta(\log n)$. Ayrıntı Bölüm 4'tedir.

Soru 3 (derste sözlü, 00:26:41)

$\sum_{i=2}^{n-1} \log_2 i^2$ toplamının büyüme derecesini bulun.

Çözüm (derste verildi): $\sum_{i=2}^{n-1} \log_2 i^2 = 2\left(\sum_{i=1}^{n} \log_2 i - \log_2 n\right)$. İlk toplam $\Theta(n \log n)$, çıkarılan terim $\Theta(\log n)$'dir. Düşük dereceli terim ve sabit atılınca sonuç $\Theta(n \log n)$ olur. Ayrıntı Bölüm 5'tedir.

Dersin Toparlaması

Bu uygulama saatinde haftanın konuları üç alıştırma üzerinde pekiştirildi. İlk iki alıştırmada rekürans bağıntıları geriye doğru yerine koyma ile çözüldü. Yöntemin adımları aynıdır: bağıntı birkaç kez kendi içine yerleştirilir, i adım sonraki genel ifade yazılır, sonra i yerine başlangıç koşuluna ulaştıran değer konur. Başlangıç koşulu olmadan bu son adım yapılamaz. Algoritma analizinde bu koşul algoritmanın durma noktasından bulunur. Her adımda sabit bir değer eklenen bağıntıda sonuç doğrusal çıkarken, her adımda n eklenen bağıntıda sonuç bir toplam formülüne, yani $\frac{n(n+1)}{2}$'ye dönüştü. $n/3$'e bölünen bağıntıda ise $n = 3^k$ kabulüyle sonuç $\log_3 n + 1$ bulundu. Üçüncü alıştırmada rekürans yoktu. Toplamın büyüme derecesi, sabitleri dışarı alarak, sınırları bilinen formülün biçimine kaydırarak ve düşük dereceli terimi eleyerek $\Theta(n \log n)$ bulundu.

Hoca haftayı şöyle özetledi: Bu hafta özyinelemeli olmayan fonksiyonların ve özyinelemeli algoritmaların matematiksel analizinin nasıl yapıldığı görüldü. Matematikte kullanılan bazı kurallar ve formüller hatırlandı. Rekürans bağıntısı, başlangıç koşulu ve temel işlem (basic operation) kavramlarına bakıldı ve bunlarla ilgili örnekler çözüldü. Önümüzdeki hafta böl ve yönet (divide and conquer) problemlerine geçilecek ve bilinen birkaç algoritma analiz edilecek. Ayrıca azalt ve yönet (decrease and conquer) yaklaşımının ne olduğu ve böl ve yönetten farkı konuşulacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.